

For this assignment, we worked with system processes, specifically retrieving information from active system processes on the local machine using Python's psutil library. Specifically, we used the psutil.process_iter() method to retrieve process information as an instance, specifying which attributes were relevant for the assignment, looping over each instance and retrieving information as a dictionary and storing each value on a local variable. Here is how that looks like on code:

```
for proc in psutil.process_iter(['pid', 'name', 'memory_percent', 'cpu_percent']):
    try:
        ## All of these will bve retrieved from the for variable 'proc' above

        ## get the PID
        pid = proc.info['pid'];

        ## get the name
        name = proc.info['name'];

        ## get the memory_percent
        mem = proc.info['memory_percent'];
```

Using a for loop, we iterated over every active process and stored the instance in “proc” and retrieved the information using proc.info[i], where i refers to each attribute wanted (pid, name and memory_percent).

For the sake of retrieving a cpu_percent that's not '0', and following psutil library documentation guidelines, cpu_percent was retrieved directly and using an interval of 0.1 for the sake of displaying cpu as the system CPU time elapsed before and after the interval (ignoring the initial 0.0 cpu_percent displayed when returned immediately). This is how that looks on code:

```
cpu = proc.cpu_percent(interval = 0.1);
```

To avoid the rare but ever-present exception where `memory_percent` and `cpu_percent` returned a “None” value, an if statement was created to turn that “None” into a float value of 0 to avoid program errors:

```
## handles exception where memory_percent returns a None value
if mem == None:
    mem = 0.0;

## handles exception where cpu_percent returns a None value
if cpu == None:
    cpu = 0.0;
```

To run the program, all that’s needed is to have the `psutil` library installed in the python environment. After that, simply by running `main.py`, the program will retrieve all active processes on the local computer. Once the program is running, the program will start to retrieve all active processes and will start displaying them line by line as follows:

PID	Name	Memory %	CPU %
0	kernel_task	0.00	0.00
1	launchd	0.00	0.00
514	logd	0.00	0.00
516	UserEventAgent	0.00	0.00

The biggest challenge faced when implementing the program was understanding the exceptions and how the `psutil` library expects you to handle them. One thing that kept causing the program to crash was the returned “None” value for `memory_percent` and `cpu_percent` on some active processes, but that was fixed when that “None” was replaced with the expected float value of 0. Another challenge was understanding how `cpu_percent` was retrieved and how the documentation specifies that to retrieve a value different from 0, we needed to retrieve the `cpu_percent` using a 0.1 interval to display cpu as the system CPU time elapsed before and after the interval.